FRONT-END WEB TECHNOLOGY

Unit 05

API Authentication and Best Practices

Complete, easy-to-understand
exam-ready study guide

Course code: 2504CS305
Coverage: Authentication • Authorization • API security • API design • API documentation

Prepared from the supplied Unit 05 presentation. Source topics are covered and expanded with clearly marked learning support.

How to use this guide

This guide follows the presentation’s sequence and explains its listed content in beginner-friendly language. It preserves the source terminology while adding short clarifications, examples, code interpretation, practice questions, and a final revision set. “Extra explanation” means supporting context added to make the source easier to learn; it is not a claim that the slide explicitly stated that detail.

Source scope: Unit 05, “Authentication and best practices for API,” Front-End Web Technology (2504CS305). The original slide deck includes some code-image/formatting gaps; where syntax was visible it is reconstructed in readable form, and any convention-sensitive details are noted.

Contents

Section	Topics
1	Learning outcome and authentication vs. authorization
2	Basic authentication and API keys
3	TLS / HTTPS and mutual TLS
4	OAuth 2.0
5	JSON Web Tokens (JWT)
6	OpenID Connect (OIDC)
7	Method comparison and choosing a method
8	Best practices for creating and consuming APIs
9	Creating API documentation
10	Exam practice and answer key
11	Final revision checklist

1. Learning outcome and core distinction

The stated learning outcome is to understand the importance of API authentication and API documentation. The unit also teaches authentication and authorization methods, security transport, API creation and consumption practices, and documentation.

Authentication and authorization

Authentication checks who a user or client is. **Authorization** checks what that authenticated party is allowed to access or do. Both protect a resource: proving identity does not automatically grant every permission.

Airport example: An identity document helps establish that you are the person named on it (authentication). A boarding pass for a particular flight shows whether you may board that flight (authorization).

Dimension	Authentication	Authorization
Main question	Who are you?	What may you access or do?
Typical mechanism	Password, security question, facial recognition, or an identity token	Policies, roles, permissions, scopes, or access token
Usual order	Usually occurs first	Usually follows successful authentication
Protocol relationship in source	Generally associated with OpenID Connect (OIDC)	Generally associated with OAuth 2.0
Example	Employee signs in to the company network	System permits that employee to view only allowed company records

Important nuance: The slide's "ID Token" versus "Access Token" row is a useful general distinction: an ID token communicates authentication/identity to a client, while an access token is used to obtain authorized access to a resource. Do not treat these tokens as interchangeable.

Exam points and common confusion

- Authentication establishes identity; authorization enforces permissions.
- Authorization without an established identity is generally not enough for user-specific access.
- A successfully authenticated user can still be denied a particular resource.
- OAuth 2.0 is authorization; OIDC adds an authentication/identity layer.

Quick check

Q: A signed-in employee requests payroll data but lacks the payroll role. What happened? **A:** Authentication succeeded; authorization denied access.

2. Basic authentication and API keys

Basic authentication

Definition: A simple method that sends a username and password in an HTTP Authorization header after encoding the credential pair with Base64. Base64 is an encoding, not encryption: anyone who obtains the encoded value can decode it. Therefore Basic authentication should be used only over HTTPS.

The usual value has the shape `Authorization: Basic <encoded-username:password>`. The source shows a JavaScript Fetch-style example:

```
fetch('apiUrl_here', { method: 'GET', headers: { 'Authorization': 'Basic ' + btoa(username + ':' + password) } });
```

How it works: (1) The client joins username and password with a colon. (2) It Base64-encodes that text. (3) It sends the result in the Authorization header. (4) The server decodes and checks the credentials. Since credentials accompany every request in the basic pattern, leakage has high impact.

Advantages: straightforward to implement; works with HTTPS. **Disadvantages:** credentials are sent on every request; unsafe without HTTPS; no built-in session management or expiration. **Source use case:** internal or low-security APIs.

Example: A small internal service may accept a dedicated service username/password over HTTPS. Avoid embedding a person's long-lived password in a browser application.

API key authentication

Definition: A unique key/token is generated by the backend and assigned to a client, which includes it with API requests. A key identifies a client or application for access control, usage tracking, or rate limiting; by itself it may not identify a human user.

```
fetch('apiUrl_here', { method: 'GET', headers: { 'Authorization': 'ApiKey ' + apiKey } });
```

How it works: The provider issues a key, the client stores it, and each request presents it. The server checks whether it is valid and applies limits or permitted access. Keys should be revocable; the source notes that revoking a key need not change the user's credentials.

Advantages: easy to implement; can be revoked independently of user credentials. **Disadvantages:** anyone who obtains an exposed key can use it; the basic key pattern has no built-in expiration or user identification. **Source use cases:** public APIs, usage tracking, and rate limiting.

Security note: "Public API" means the API is available to outside developers; it does not mean a secret key is safe to publish in client-side code. The source also warns consumers never to expose API keys in client code or public repositories.

Feature	Basic authentication	API key
Credential presented	Username/password pair, Base64-encoded	Issued client key/token
Sent repeatedly?	Typically with every request	Typically with each request
Revocation	Changing credentials may affect their use elsewhere	Key can be revoked separately
Main risk	Credential disclosure, especially without HTTPS	Key theft/exposure
Source use case	Internal or low-security APIs	Public APIs, tracking, rate limits

Common mistakes

- Calling Base64 "encryption." It is reversible encoding.
- Sending Basic credentials or API keys over plain HTTP.

- Putting a secret key in browser JavaScript, a public repository, or a screenshot.
- Assuming an API key necessarily proves the identity of an individual user.

3. TLS encryption

TLS (Transport Layer Security) protects communication between a client and server while data travels over the network. A website or API uses HTTPS when HTTP communication is protected by TLS. The certificate helps the client verify the server's identity; certificates are issued through Certificate Authorities (CAs), as described in the source.

Transport encryption is not application authorization. TLS protects data in transit and helps prevent interception or tampering by a network attacker, but it does not decide whether a caller is allowed to read a particular API resource. Pair it with an authentication/authorization method.

Typical connection in simple steps

- The client connects to an HTTPS endpoint.
- The server presents its TLS certificate.
- The client checks that the certificate is trusted and matches the server it intended to reach.
- The client and server establish encryption keys and exchange protected traffic.
- The API then separately applies its authentication and authorization rules.

Mutual TLS (mTLS)

In ordinary TLS, the server proves its identity to the client. In **mutual TLS**, both server and client present certificates so each side can verify the other. It is commonly useful for tightly controlled service-to-service connections.

Advantages: encrypts data in transit and helps defend against man-in-the-middle attacks; works alongside other authentication methods. **Disadvantages:** does not handle application authentication/authorization on its own; certificate issuance, renewal, and management can be complex. **Source use cases:** banking, healthcare, and sensitive APIs.

Exam distinction

Question	Answer
Does TLS encrypt data in transit?	Yes.
Does TLS alone decide the user's API permissions?	No.
What does mTLS add?	Certificate-based verification of both client and server.
What does HTTPS indicate?	HTTP communication protected using TLS.

4. OAuth 2.0

OAuth 2.0 (Open Authorization) is a delegated authorization framework. It lets an application obtain limited access to a user's resources at another service with the user's consent, without the user sharing their password with that application. OAuth 2.0 is primarily authorization—not, by itself, a user-authentication protocol.

Example: A photo-printing service asks permission to read selected photos from a storage service. The user grants consent at the storage provider. The application receives an access token with permitted scope rather than the user's password.

Conceptual flow

- A client application requests access to a resource.
- The user is sent to the authorization provider and signs in there, if needed.
- The user reviews and grants or denies consent.
- The authorization server issues an access token according to the grant and permissions.
- The client sends the access token to the API, commonly as a Bearer token.
- The API validates the token and enforces its permissions/scopes.

```
fetch('apiUrl_here', { method: 'GET', headers: { 'Authorization': 'Bearer ' + accessToken } });
```

Terms: A *client* is the application requesting access. A *resource owner* is typically the user who controls the data. An *authorization server* issues tokens. A *resource server* hosts the protected API. A *scope* describes the access requested or granted.

Advantages: supports consented third-party access and scoped permissions; avoids sharing user credentials with the client app. **Disadvantages:** more complex to implement and requires an authorization server. **Source use cases:** third-party integrations and enterprise applications. The source also lists social logins, but for login/authentication the relevant companion standard is usually OIDC.

Critical distinction: OAuth access tokens authorize access to a resource. A successful OAuth flow may involve the user authenticating at a provider, but OAuth itself does not standardize an identity assertion to the client. Use OIDC when the client needs a standardized login/identity result.

Common mistakes

- Saying "OAuth 2.0 is authentication." Its primary job is delegated authorization.
- Treating a Bearer token as harmless: whoever possesses it may be able to use it within its permissions.
- Granting broader scopes or longer-lived access than the application needs.

5. JSON Web Tokens (JWT)

JSON Web Token (JWT) is a compact, self-contained token format for transmitting claims (statements about a subject or token) between parties as a JSON object. RFC 7519 defines JWT. A signed JWT can be checked for integrity and authenticity: the receiver can detect if signed content changed and verify that it was produced by someone holding the relevant signing key.

Important: signing is not the same as encryption. A typical signed JWT's header and payload are encoded, not hidden. Do not put secrets or sensitive personal data in a JWT payload unless an encryption mechanism is explicitly used.

Three parts

Part	What it contains / does
Header	Metadata such as token type and signing algorithm.
Payload	Claims/data, for example subject, issuer, or expiry. Claims are not automatically confidential.
Signature	Cryptographic value used to detect tampering and validate the token under the chosen key/algorithm.

A compact token has three Base64URL-encoded segments separated by dots:

```
header.payload.signature
xxxxxxxxxx.yyyyyyyy.zzzzzzzzzzzzzzzz
```

Signing choices and use

The source says JWTs may be signed with a shared secret using HMAC, or with public/private key pairs using RSA or ECDSA. With HMAC, parties that verify also share the secret. With an asymmetric signature, the issuer signs with a private key and verifiers can validate using the public key.

JWTs are commonly used for authorization: after login, later requests carry a token, often in `Authorization: Bearer <token>`, and services check token claims. JWTs can also support information exchange because a signature can help verify the issuer and detect payload tampering. The source also highlights single sign-on (SSO), compactness, and use across domains.

```
fetch('apiUrl_here', { method: 'GET', headers: { 'Authorization': 'Bearer ' + token } });
```

What "stateless" means

A server can validate a self-contained token without looking up a per-request session record, depending on system design. This can make distributed services easier to scale. However, immediate revocation is harder unless the system adds measures such as short expiration, revocation lists, or key rotation. JWTs do not make an API secure automatically; validation rules, key handling, expiry, issuer, audience, and permissions still matter.

Common mistakes

- Assuming every JWT is signed or encrypted in the same way; inspect the expected format and validation rules.
- Trusting a token merely because it is syntactically a JWT; verify its signature and relevant claims.
- Putting passwords or confidential data in an ordinary signed payload.
- Confusing the token's format (JWT) with a protocol that issues or uses it (for example, OIDC).

Exam memory aid: JWT = header + payload + signature. The signature protects integrity; it does not by itself conceal the payload.

6. OpenID Connect (OIDC)

OpenID Connect is an identity layer built on OAuth 2.0. OAuth provides authorization mechanics; OIDC adds a standardized way for a client to authenticate a user and receive identity information, including an ID Token (in JWT format).

Basic sign-in flow

- The client redirects the user to an OpenID Provider (OP), such as Google or Microsoft.
- The provider authenticates the user and applies its sign-in process.
- The provider returns an ID Token to the client, typically alongside OAuth authorization results.
- The client validates the token and uses the identity result to establish a sign-in session.

ID token: tells the client about the authentication event and user identity. **Access token:** is presented to a resource/API for authorized access. The client should validate tokens according to the protocol and provider metadata rather than trusting arbitrary token text.

Advantages: provides a standardized authentication layer where OAuth 2.0 alone only handles authorization; supports social sign-in. **Disadvantages:** requires an identity provider and can be complex to integrate.

Feature	OAuth 2.0	OIDC
Primary purpose	Delegated authorization	Authentication/identity on top of OAuth 2.0
Typical token of interest	Access token	ID Token, usually JWT; may also receive access token
Question answered	What API/resource access is allowed?	Who signed in?
Example	Allow a calendar app to read events	Sign in to a site using an identity provider

Likely exam trap: OIDC is not a replacement for OAuth; it builds on OAuth 2.0 and adds identity/authentication semantics.

7. Method comparison and selection

Method	Primary category	Source security rating	Stateless in source?	Source use case
Basic Auth	Authentication	Low	No	Internal APIs
API key	Client/API access credential	Medium	Yes	Public APIs, rate limiting
TLS	Transport security	High	—	Secure communication
OAuth 2.0	Authorization	High	Yes	Third-party access, enterprise apps
JWT	Token format used for authentication/authorization	High	Yes	Microservices, SSO, API security
OIDC	Authentication and identity	High	Yes	SSO, federated identity

The ratings and “stateless” entries above reproduce the slide’s summary table, not a universal ranking. Actual security depends on configuration, implementation, threat model, and operational controls. TLS is a transport layer, while the other entries primarily describe credentials, tokens, or protocol functions, so their categories are not directly equivalent.

A practical mental model

- **Protect the network path:** use HTTPS/TLS.
- **Identify a user:** use an authentication mechanism; OIDC is a common federated identity layer.
- **Grant delegated API access:** use OAuth 2.0 with appropriate scopes.
- **Represent claims compactly:** JWT is one possible token format, not a complete authorization system by itself.
- **Identify/limit a calling application:** an API key may help with access management, tracking, or rate limits, but protect and rotate it.
- **Simple legacy/internal credential pattern:** Basic authentication only over HTTPS and with careful credential handling.

One-minute comparison

Authentication asks “who?” Authorization asks “what can they do?” TLS asks “is the communication protected in transit?” OAuth asks “can this client access these resources with granted permission?” OIDC asks “who signed in?” JWT answers “what compact signed token format carries claims?”

8. API best practices

Creating APIs

RESTful principles

- Use HTTP methods appropriately: GET to retrieve, POST to create/submit, PUT to replace or update according to the API's contract, and DELETE to remove.
- Use clear, logical, resource-based URLs. Name resources consistently and avoid confusing endpoint structures.
- Use HTTP status codes to communicate success or failure. Pair codes with useful, consistent response bodies where appropriate.

Consistency, versions, and documentation

- Use clear, descriptive names for endpoints, parameters, and fields.
- Choose and consistently apply a convention such as snake_case or camelCase.
- Version the API (for example, /v1/ and /v2/) so changes can be introduced without unexpectedly breaking existing clients.
- Publish complete, current documentation with a tool such as Swagger/OpenAPI; maintain a changelog.

Errors and security

- Return meaningful, consistent errors with suitable HTTP status codes; avoid exposing internal secrets or stack traces.
- Log errors for monitoring and analysis while protecting sensitive information in logs.
- Implement authentication and authorization, such as OAuth 2.0, JWT-based systems, or API keys where suitable.
- Use HTTPS to encrypt traffic.
- Validate and sanitize inputs to reduce injection and malformed-data risks.
- Apply rate limits to reduce abuse and protect service capacity.
- Protect sensitive data throughout storage, transport, and logging.

Performance and testing

- Reduce unnecessary data transfer and processing; choose suitable formats such as JSON or Protocol Buffers.
- Use caching when responses can safely be reused.
- Paginate large result sets rather than returning everything in one response.
- Create unit tests for small components, integration tests for connected components, and end-to-end tests for whole user/API flows.
- Use Postman or curl for manual testing and automate repeatable tests.

Consuming APIs

- Read the documentation for endpoints, parameters, authentication, and expected behavior; test endpoints with Postman or curl.
- Use a reliable HTTP client library and implement the documented authentication method correctly.
- Handle HTTP and network errors gracefully; parse the response format correctly, such as JSON.
- Retry transient failures carefully. Avoid retrying non-repeatable operations blindly; use limits/backoff and respect server guidance.
- Respect rate limits and keep API keys out of browser code and public repositories.
- Use asynchronous requests where suitable to improve responsiveness/performance.
- Validate returned data against application expectations; do not assume every response is well-formed.

- Cache responses when appropriate to reduce unnecessary calls; log requests/responses carefully for debugging and monitoring.

Example consumer checklist: read the API contract → store credentials safely → send a request over HTTPS → check status and response shape → handle errors and limits → validate data → log only what is safe → cache or retry only when appropriate.

Why these practices matter

Consistent contracts make APIs easier to learn and less likely to break clients. Security controls limit unauthorized access and abuse. Tests catch regressions; pagination, caching, and efficient payloads help performance. Good consumers remain robust when networks fail, responses change within the contract, or rate limits apply.

9. Creating API documentation

API documentation is a set of instructions that explains how developers and other users can use an API and its services. It helps developers understand how to make calls and build integrations. It commonly includes code examples, tutorials, parameter details, and input/return formats.

Documentation may be written manually or generated/maintained using tools such as Swagger/OpenAPI or Postman. A useful document starts with a high-level overview and then describes each endpoint.

High-level overview should include

- What the API is for and its base URL.
- How authentication works and how credentials/tokens are supplied.
- Rate limits and usage policies.

For each endpoint, document

Item	What to explain
HTTP method	The action/verb used, such as GET or POST.
URL	The path and any path variables.
Description	What the endpoint does and who may use it.
Request parameters	Names, locations, types, required/optional status, allowed values, and examples.
Example response	Representative response body and relevant status codes/format.

Additional exam-ready documentation quality checks

- Show the authentication header or flow without publishing real secrets.
- Explain common errors and status codes, pagination, rate limits, and versioning when applicable.
- Keep examples valid and update documentation when behavior changes; record changes in a changelog.

Sample endpoint description: `GET /v1/books/{bookId}` — retrieves one book. Document the path parameter `bookId`, required authentication, possible success response, and errors such as not found or unauthorized. This is an illustrative example, not an endpoint from the source slides.

Why documentation matters

Clear documentation reduces guesswork, helps consumers call the service correctly, makes integrations faster, and aligns the API's intended contract with its actual behavior. The unit learning outcome explicitly identifies understanding API documentation as important.

10. Exam practice

These questions test the material covered in this guide. They are practice questions, not predictions about a particular exam.

Short-answer questions with answers

1. Define authentication. The process of verifying who a user or client is.

2. Define authorization. The process of deciding what an authenticated user/client may access or do.

3. Why must Basic authentication use HTTPS? Base64 is encoding, not encryption; HTTPS protects credentials while they travel over the network.

4. Give two disadvantages of API keys from the source. An exposed key can be used by anyone; the basic API-key pattern has no built-in expiration or user identification.

5. What does TLS do, and what does it not do alone? TLS encrypts communication in transit and helps protect the connection; it does not by itself make application authorization decisions.

6. Is OAuth 2.0 an authentication protocol? No. It is primarily a delegated authorization framework.

7. Name the three JWT parts. Header, payload, and signature.

8. What does OIDC add to OAuth 2.0? An identity layer for authentication, including an ID Token.

9. Name three items to document for each endpoint. HTTP method, URL, description, request parameters, and example response are source-listed items; any three suffice.

10. What is pagination for? It divides large result sets into smaller pages rather than returning all records at once.

Long-answer questions with structured answers

1. Compare authentication and authorization.
• Define each: authentication verifies identity; authorization checks allowed actions/resources.
• Give their usual order: authentication normally precedes authorization.
• Explain that a known identity does not imply full access.
• Use the airport example: ID checks identity; boarding pass/gate permission determines boarding.

2. Explain JWT structure and uses.
• JWT is a compact token format defined in RFC 7519 for JSON claims.
• It consists of header, payload, and signature, separated by dots.
• It can be signed with HMAC or an asymmetric RSA/ECDSA key pair.
• A receiver validates the signature to detect tampering; ordinary signed JWT payloads are not automatically secret.
• Uses include carrying authorization claims and information exchange; requests often send it as a Bearer token.

3. Describe API creation best practices.
• Follow resource-based REST conventions and use HTTP methods/status codes appropriately.
• Keep naming consistent; version APIs and maintain documentation/changelog.
• Provide consistent errors and safe logging.
• Secure the API with suitable authentication/authorization, HTTPS, input validation, rate limiting, and sensitive-data protection.
• Improve performance through efficient payloads, caching, and pagination.
• Test with unit, integration, and end-to-end tests; automate testing.

4. Explain the difference between OAuth 2.0 and OIDC.
• OAuth 2.0 grants delegated, scoped access to resources using access tokens.
• OAuth alone does not standardize user authentication to the client.
• OIDC adds identity/authentication on top of OAuth 2.0 and supplies an ID Token.
• OAuth asks what access is granted; OIDC helps answer who signed in.

5. Outline a robust API-consumer workflow.
• Read documentation; test endpoints.
• Use a dependable HTTP client and correctly handle credentials.

• Parse and validate responses; manage errors and transient network failures.
• Respect rate limits, protect API keys, and use asynchronous calls where suitable.
• Cache selectively and log safely for monitoring/debugging.

MCQs

1. Which statement best describes authentication?
A. Deciding which records a user may read B. Verifying who a user is C. Encrypting network traffic D. Paginating results

2. Basic authentication commonly encodes credentials using:
A. AES B. Base64 C. JWT D. TLS

3. Which method protects data in transit?
A. TLS B. API documentation C. Pagination D. Naming convention

4. OAuth 2.0 primarily provides:
A. Authorization B. A database C. Transport encryption D. API versioning

5. A JWT consists of:
A. Username, password, and API key B. Header, payload, signature C. URL, method, status D. Client, server, certificate

6. Which JWT statement is correct?
A. A signature always hides the payload B. Base64URL encoding guarantees confidentiality C. A signature can help detect tampering D. Every JWT is an OAuth protocol

7. OIDC adds which capability to OAuth 2.0?
A. Identity/authentication layer B. HTTP caching C. Pagination D. TLS certificate issuance

8. Which is a good API practice?
A. Use inconsistent endpoint names B. Return all records always C. Apply rate limits and validate inputs D. Publish secret API keys

9. What is the purpose of API versioning?
A. To prevent any authentication B. To evolve the API while reducing breakage for existing clients C. To encode passwords D. To replace HTTPS

10. Which item is expected in endpoint documentation?
A. HTTP method and request parameters B. Developer’s private password C. Unrelated source code D. No response example

11. Final revision

Chapter-wise recap

- **Authentication/authorization:** verify identity (“who?”) then decide permissions (“what?”).
- **Basic authentication:** Base64-encoded username:password in Authorization; use HTTPS; credentials typically accompany each request.
- **API keys:** backend-issued client credential; easy to revoke separately; protect against exposure; useful for tracking and rate limiting.
- **TLS/mTLS:** encrypt transport; certificates support identity checks; mTLS verifies both client and server; TLS is not API permission logic.
- **OAuth 2.0:** consented, scoped, delegated authorization; access tokens; not authentication by itself.
- **JWT:** compact claims format: header.payload.signature; signed tokens can be checked for tampering; signing does not mean secrecy.
- **OIDC:** OAuth 2.0 identity layer; ID Token communicates identity/authentication to client.
- **API design:** RESTful methods/URLs/status codes, consistent naming, versioning, docs, errors, security, performance, tests.
- **API consumption:** understand docs; authenticate correctly; handle errors/retries/rate limits; parse and validate data; protect keys; cache/log thoughtfully.
- **Documentation:** overview with base URL, auth, rate limits/policies; per endpoint method, URL, purpose, parameters, example response.

Essential definitions

Term	Memorize
Authentication	Verifying who a user/client is.
Authorization	Determining what that user/client may access or do.
TLS	Protocol that protects communication in transit through encryption and certificate-based trust.
OAuth 2.0	Framework for delegated, consent-based authorization to resources.
JWT	Compact JSON claims token format with header, payload, and signature.
OIDC	Identity/authentication layer built on OAuth 2.0.
API documentation	Instructions describing how developers use an API and its endpoints.

Key comparisons

Do not confuse	Difference
Authentication vs authorization	Identity check vs permission check.
OAuth vs OIDC	Delegated access vs identity/authentication layer on OAuth.
ID token vs access token	Identity information for client vs permission to access a resource.
Encoding vs encryption	Encoding changes representation; encryption protects confidentiality with cryptographic keys.
JWT signing vs encryption	Signing helps verify integrity/authenticity; encryption conceals content.
TLS vs API authentication	Transport protection vs caller identity/permission checks.

Syntax to recognize

```
Authorization: Basic
Authorization: ApiKey
Authorization: Bearer
```

Last-minute checklist

- Can I explain who authentication verifies and what authorization decides?
- Can I state why Base64 does not make Basic credentials secret?
- Can I explain TLS, HTTPS, and mTLS—and what TLS does not do?
- Can I distinguish OAuth authorization from OIDC authentication?
- Can I label JWT header, payload, and signature and explain the signature's limits?
- Can I name the API creation and consumption best practices?
- Can I list the information required for an API overview and each endpoint?

MCQ answer key

Q	Answer	Reason
1	B	Authentication verifies identity.
2	B	Base64 is the encoding used in Basic auth.
3	A	TLS protects communication in transit.
4	A	OAuth 2.0 is authorization.
5	B	JWT has header, payload, and signature.
6	C	Signatures help detect tampering, not hide payloads.
7	A	OIDC adds identity/authentication.
8	C	Input validation and rate limits are good practice.
9	B	Versioning supports API evolution with less client disruption.
10	A	Endpoint docs include method and request parameters.

Coverage check: The supplied presentation's outline and all readable topic areas are represented: importance and distinction of authentication/authorization; Basic auth; API keys; TLS/mTLS; OAuth 2.0; JWT; OIDC; comparison table; creating/consuming API best practices; and creating API documentation. Slides with visually embedded code were reconstructed from readable extracted text; the presentation's decorative images do not add readable study claims in the extracted content.